

UT 2.2 · JAVASCRIPT

Métodos de arrays

Agregar, buscar, recorrer y transformar datos con
ECMAScript 6

push

find

map

reduce

```
    [...].push()  
    .filter(x => x)  
    .map(x => x * 2)  
    .reduce((a, b) => a + b)
```

Cinco familias de métodos



Agregar y quitar

`push` •
`pop` •
`shift` •
`unshift` •
`splice` •
`slice`
`concat`



Recorrer

`forEach`



Buscar

`indexOf`
`lastIndexOf`
`includes`
`find`
`findIndex`
`findLastIndex`
`filter`



Transformar

`map`
`sort` •
`reverse` •
`split` / `join`
`reduce`
`reduceRight`



Extra

`Array.isArray`
`some`
`every`
`fill` •
`flat`

- Modifica (muta) el array original. El resto devuelve un valor o un array **nuevo**.

AGREGAR Y QUITAR

Repaso: los extremos del array

unshift(...) →

← shift()

'café'

'pan'

'leche'

'huevos'

← push(...)

pop() →

```
const compra = ['pan', 'leche'];

compra.push('huevos');    // ['pan', 'leche', 'huevos']
compra.unshift('café');   // ['café', 'pan', 'leche', 'huevos']

const ultimo = compra.pop();    // 'huevos'
const primero = compra.shift(); // 'café'
console.log(compra);            // ['pan', 'leche']
```

AGREGAR Y QUITAR

¿Y si queremos borrar del medio?

Los arrays son objetos, así que podríamos probar con **delete**.



length sigue siendo 3

Borra el **valor**, pero deja el hueco: el resto de elementos no se desplaza.

Necesitamos un método que reordene el array: **splice**.

```
const frase = ['voy', 'a', 'casa'];

delete frase[1];           // borra 'a'

console.log(frase[1]);     // undefined
console.log(frase);        // ['voy', <vacío>, 'casa']
console.log(frase.length); // 3
```

Eliminar, reemplazar y recoger lo eliminado

```
// 1) Eliminar: desde el índice 1, borra 1 elemento
const lenguaje = ['Yo', 'estudio', 'JavaScript'];
lenguaje.splice(1, 1);
console.log(lenguaje); // ['Yo', 'JavaScript']

// 2) Reemplazar: borra los 3 primeros e inserta 2 nuevos
const frase = ['Yo', 'estudio', 'JavaScript', 'ahora', 'mismo'];
frase.splice(0, 3, 'a', 'bailar');
console.log(frase); // ['a', 'bailar', 'ahora', 'mismo']

// 3) Recoger lo eliminado: splice devuelve un array
const otra = ['Yo', 'estudio', 'JavaScript', 'ahora', 'mismo'];
const eliminados = otra.splice(0, 2);
console.log(eliminados); // ['Yo', 'estudio']
```

Insertar sin borrar e índices negativos

`cantidadABorrar = 0` → solo inserta

```
const palabras = ['Yo', 'estudio', 'JavaScript'];  
// desde el índice 2, borra 0 e inserta 3 elementos  
palabras.splice(2, 0, 'el', 'complejo', 'lenguaje');  
console.log(palabras); // ['Yo', 'estudio', 'el', 'complejo', 'lenguaje', 'JavaScript']
```

Índices negativos: se cuentan desde el final

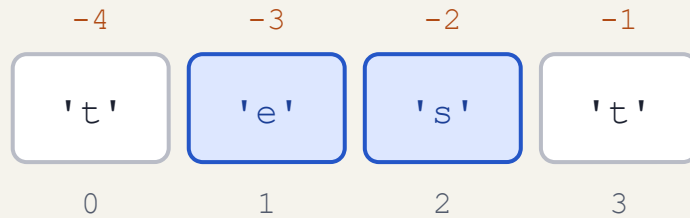
```
const numeros = [1, 2, 5];  
// desde -1 (una posición antes del final),  
// borra 0 e inserta 3 y 4  
numeros.splice(-1, 0, 3, 4);  
console.log(numeros); // [1, 2, 3, 4, 5]
```



AGREGAR Y QUITAR · SLICE

slice: copiar un trozo sin tocar el original

```
arr.slice(inicio, fin) // fin NO incluido
```



`slice(1, 3) → ['e', 's']`

```
const letras = ['t', 'e', 's', 't'];

console.log(letras.slice(1, 3)); // ['e', 's']
console.log(letras.slice(-2));  // ['s', 't']

const copia = letras.slice();    // copia completa
copia.push('!');
console.log(letras); // ['t', 'e', 's', 't'] intacto
```

Es como **str.slice** de los strings, pero genera sub-arrays. Sin argumentos crea una **copia** del array.

RECORRER · FOREACH

forEach: ejecutar una función por cada elemento

```
arr.forEach((elemento, indice, array) => { ... })
```

```
const personajes = ['Bilbo', 'Gandalf', 'Nazgul'];

// Solo necesitamos el elemento
personajes.forEach((personaje) => console.log(personaje));

// Con el índice y el array completo
personajes.forEach((personaje, indice, lista) => {
  console.log(`${personaje} está en la posición ${indice} de [${lista}]`);
});

// Bilbo está en la posición 0 de [Bilbo,Gandalf,Nazgul] ...
```

⚠️ **forEach no devuelve nada** (undefined): si necesitas un array de resultados, usa **map**.

BUSCAR

indexOf, lastIndexOf e includes

indexOf(elem, desde)

Primera posición donde está, o -1.

lastIndexOf(elem)

Igual, pero busca de derecha a izquierda.

includes(elem, desde)

true o **false**. Úsalo si solo te importa si existe.

```
const valores = [1, 0, false];
console.log(valores.indexOf(0));      // 1
console.log(valores.indexOf(false));  // 2 (usa ===)
console.log(valores.indexOf(null));   // -1
console.log(valores.includes(1));     // true
```

```
const frutas = ['Manzana', 'Naranja', 'Manzana'];
console.log(frutas.indexOf('Manzana')); // 0
console.log(frutas.lastIndexOf('Manzana')); // 2
```

```
const raros = [NaN];
console.log(raros.indexOf(NaN)); // -1 (fallo)
console.log(raros.includes(NaN)); // true (bien)
```

BUSCAR · FIND

find, findIndex y findLastIndex: buscar con una condición

```
arr.find((elem) => condición)
```

find devuelve el **primer elemento** que cumple la condición, o **undefined**.

findIndex devuelve su **índice**, o **-1**.

findLastIndex busca desde el final.

Ideal para arrays de objetos.

```
const usuarios = [
  { id: 1, nombre: 'Celina' },
  { id: 2, nombre: 'David' },
  { id: 3, nombre: 'Federico' },
  { id: 4, nombre: 'Celina' },
];

const usuario = usuarios.find((u) => u.id === 1);
console.log(usuario.nombre); // 'Celina'
console.log(usuarios.find((u) => u.id === 9)); // undefined

const esCelina = (u) => u.nombre === 'Celina';
console.log(usuarios.findIndex(esCelina)); // 0
console.log(usuarios.findLastIndex(esCelina)); // 3
```

BUSCAR · FILTER

find encuentra uno; filter, todos los que cumplen

puntos

4

9

12

7

15

condición: `n > 8`

```
.find((n) => n > 8)
```

9

un elemento (o undefined)

```
.filter((n) => n > 8)
```

9

12

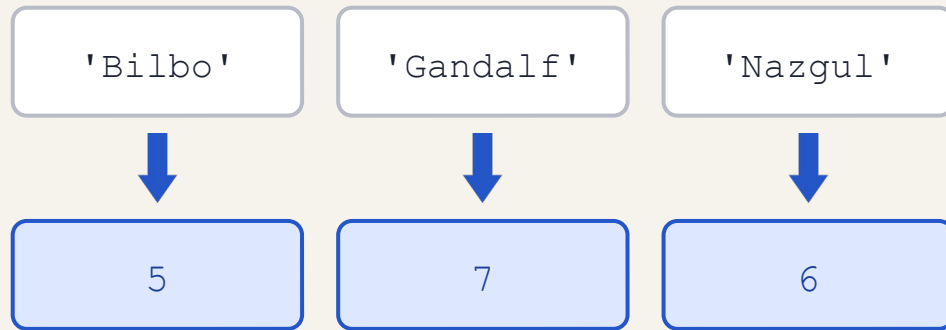
15

un array nuevo

```
const puntos = [4, 9, 12, 7, 15];  
const primero = puntos.find((n) => n > 8);    // 9  
const todos = puntos.filter((n) => n > 8);     // [9, 12, 15]  
const ninguno = puntos.filter((n) => n > 99);  // [] (array vacío)
```

TRANSFORMAR · MAP

map: un array nuevo con cada elemento transformado



`(nombre) => nombre.length`

La función se aplica a **cada** elemento y su resultado ocupa la misma posición en el array nuevo.

Mismo número de elementos. El original no cambia.

```
const personajes = ['Bilbo', 'Gandalf', 'Nazgul'];  
const longitudes = personajes.map((nombre) => nombre.length);  
console.log(longitudes); // [5, 7, 6]
```

```
const precios = [10, 20, 30];  
const conIva = precios.map((precio) => precio * 1.21);  
console.log(conIva); // [12.1, 24.2, 36.3]
```

TRANSFORMAR · SORT

```
[1, 2, 15].sort()
```

→ [1, 15, 2] ¿?

Por defecto, **sort convierte los elementos a string** y los compara en orden lexicográfico: '15' va antes que '2'.

TRANSFORMAR · SORT

sort(fn): le decimos cómo comparar

La función recibe dos elementos **a** y **b** y devuelve un número:

Devuelve	Resultado
positivo	b va antes que a
negativo	a va antes que b
0	son iguales

Por eso **a - b** ordena de menor a mayor y **b - a** de mayor a menor.

```
const numeros = [1, 2, 15];

// Versión larga
numeros.sort(function (a, b) {
  if (a > b) return 1;
  if (a === b) return 0;
  return -1;
});

// Versión corta con función flecha
numeros.sort((a, b) => a - b); // [1, 2, 15]
numeros.sort((a, b) => b - a); // [15, 2, 1]
```

Ordenar textos con acentos e invertir el orden

```
str.localeCompare

const paises = ['Österreich', 'Andorra', 'Vietnam'];

// Compara códigos de carácter: incorrecto
paises.sort((a, b) => (a > b ? 1 : -1));
// ['Andorra', 'Vietnam', 'Österreich']

// Compara según el idioma: correcto
paises.sort((a, b) => a.localeCompare(b));
// ['Andorra', 'Österreich', 'Vietnam']
```

```
arr.reverse()

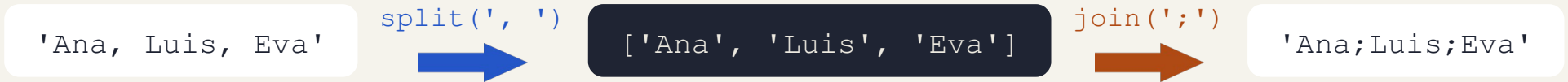
const cuenta = [1, 2, 3, 4, 5];

cuenta.reverse();

console.log(cuenta);
// [5, 4, 3, 2, 1]
```

⚠️ sort y reverse **modifican** el array y además lo devuelven. Para no perder el original: [\[...numeros\].sort\(...\)](#)

split y join: de string a array y viceversa



```
const destinatarios = 'Ana, Luis, Eva';
const lista = destinatarios.split(', ');

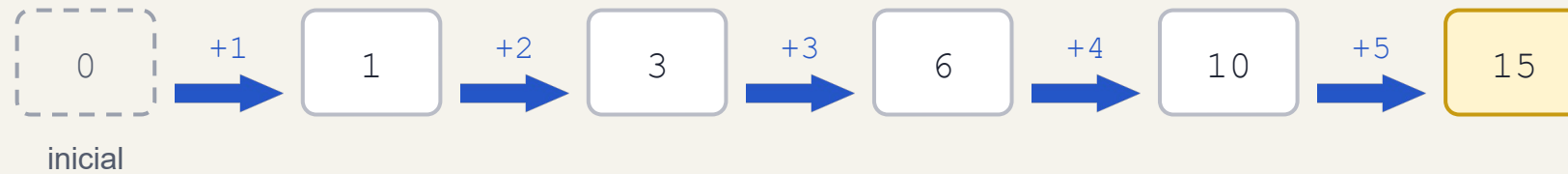
for (const nombre of lista) {
  console.log(`Un mensaje para ${nombre}.`);
}

console.log('a, b, c, d'.split(', ', 2)); // ['a', 'b'] (límite)
console.log('test'.split(''));           // ['t', 'e', 's', 't']
console.log(lista.join(';'));             // 'Ana;Luis;Eva'
```

TRANSFORMAR · REDUCE

reduce: de un array a un único valor

```
arr.reduce((acumulador, elemento, indice, array) => ..., valorInicial)
```



Como una bola de nieve: el resultado de cada llamada se pasa como **acumulador** a la siguiente, y el último es el resultado final.

```
const numeros = [1, 2, 3, 4, 5];  
const total = numeros.reduce((suma, actual) => suma + actual, 0);  
console.log(total); // 15
```

TRANSFORMAR · REDUCE

Paso a paso: (suma, actual) => suma + actual

llamada	suma	actual	resultado
1. ^a llamada	0	1	1
2. ^a llamada	1	2	3
3. ^a llamada	3	3	6
4. ^a llamada	6	4	10
5. ^a llamada	10	5	15

Cada **resultado** se convierte en la **suma** de la fila siguiente.

En la primera llamada, suma vale el valor inicial: **0**.

Sin valor inicial... con cuidado. Y reduceRight

```
const numeros = [1, 2, 3, 4, 5];
numeros.reduce((suma, actual) => suma + actual); // 15: empieza con el 1

const vacio = [];
vacio.reduce((suma, actual) => suma + actual);
// TypeError: Reduce of empty array with no initial value
vacio.reduce((suma, actual) => suma + actual, 0); // 0

// reduceRight: igual, pero de derecha a izquierda
['a', 'b', 'c'].reduceRight((texto, letra) => texto + letra, ''); // 'cba'
```



Especifica siempre el valor inicial. Sin él, reduce usa el primer elemento y, si el array está vacío, lanza un error.

EXTRA · ARRAY.ISARRAY

¿Es un array? typeof no sirve

```
console.log(typeof {}); // 'object'
console.log(typeof []); // 'object' (!igual!)

console.log(Array.isArray({})); // false
console.log(Array.isArray([])); // true
```

Los arrays no son un tipo distinto: **están basados en objetos**.

Para distinguirlos usamos el método estático **`Array.isArray(valor)`**.

EXTRA · THISARG

El parámetro opcional thisArg

```
arr.filter(fn, thisArg)
```

find, filter, map... (todos salvo sort)
aceptan un último argumento: el valor
de **this** dentro de la función.

Se usa poco: una función flecha hace lo
mismo de forma más clara.

```
const ejercito = {
  edadMinima: 18,
  edadMaxima: 27,
  puedeUnirse(recluta) {
    return recluta.edad >= this.edadMinima &&
      recluta.edad < this.edadMaxima;
  },
};
const reclutas = [{ edad: 16 }, { edad: 20 }, { edad: 30 }];

reclutas.filter(ejercito.puedeUnirse, ejercito); // [{ edad: 20 }]
// Equivalente y más habitual:
reclutas.filter((r) => ejercito.puedeUnirse(r));
// Sin thisArg: this sería undefined → error
```

EXTRA

Otros métodos útiles

`some / every: ¿alguno? ¿todos?`

```
const edades = [12, 17, 25];  
edades.some((e) => e >= 18); // true  
edades.every((e) => e >= 18); // false
```

`every para comparar arrays`

```
const sonIguales = (a, b) =>  
  a.length === b.length &&  
  a.every((valor, i) => valor === b[i]);  
sonIguales([1, 2], [1, 2]); // true
```

`fill(valor, inicio, fin)`

```
new Array(3).fill(0); // [0, 0, 0]  
[1, 2, 3, 4].fill(9, 1, 3); // [1, 9, 9, 4]
```

`flat(profundidad) / flatMap(fn)`

```
[1, [2, [3, [4]]]].flat(); // [1, 2, [3,  
[4]]]  
[1, [2, [3, [4]]]].flat(Infinity); // [1, 2, 3, 4]  
['hola mundo'].flatMap((f) => f.split(' '));  
// ['hola', 'mundo']
```

IDEA CLAVE

¿Modifica el original o devuelve algo nuevo?

Mutan el array

push pop shift unshift splice sort
reverse fill copyWithin

Devuelven algo nuevo

slice concat map filter find
includes reduce join flat some

Truco: ordenar sin perder el original

```
const numeros = [3, 1, 2];  
const ordenados = [...numeros].sort((a, b) => a - b); // copia y ordena  
const ordenados2 = numeros.toSorted((a, b) => a - b); // ES2023: sin mutar  
console.log(numeros); // [3, 1, 2] intacto
```

Organiza tus utilidades de arrays en un módulo

utilidades/arrays.js

```
export const sumar = (numeros) =>
  numeros.reduce((total, n) => total + n, 0);

export const media = (numeros) =>
  numeros.length === 0
    ? 0
    : sumar(numeros) / numeros.length;

export const aprobados = (notas) =>
  notas.filter((nota) => nota >= 5);

export const ordenar = (numeros) =>
  [...numeros].sort((a, b) => a - b);
```

main.js

```
import { media, aprobados, ordenar }
  from './utilidades/arrays.js';

const notas = [7, 4.5, 9, 3, 6];

console.log(media(notas));      // 5.9
console.log(aprobados(notas)); // [7, 9, 6]
console.log(ordenar(notas));    // [3, 4.5, 6, 7, 9]
console.log(notas);             // sin cambios

// En index.html:
// <script type="module" src="main.js"></script>
```

RESUMEN

Chuleta de métodos de arrays

Método	Qué hace
push(...items)	Añade al final (muta)
pop()	Quita el último (muta)
shift()	Quita el primero (muta)
unshift(...items)	Añade al inicio (muta)
splice(i, n, ...)	Borra e inserta (muta)
slice(inicio, fin)	Copia un trozo
concat(...items)	Une en un array nuevo
forEach(fn)	Recorre, no devuelve nada
indexOf / includes	Posición / true o false
find / findIndex	Primer elemento / índice
filter(fn)	Todos los que cumplen

Método	Qué hace
map(fn)	Transforma cada elemento
sort(fn)	Ordena (muta)
reverse()	Invierte (muta)
split / join	String ↔ array
reduce(fn, inicial)	Calcula un único valor
Array.isArray(v)	¿Es un array?
some / every	¿Alguno? / ¿Todos?
fill(valor)	Rellena (muta)
flat / flatMap	Aplana arrays anidados

PRÁCTICA

Reto: encadena métodos

productos.js

```
const productos = [
  { nombre: 'Teclado', precio: 25, stock: 4 },
  { nombre: 'Ratón', precio: 12, stock: 0 },
  { nombre: 'Monitor', precio: 180, stock: 2 },
  { nombre: 'Casco', precio: 40, stock: 7 },
];
```

```
// Ejemplo de encadenamiento:
const baratos = productos
  .filter((p) => p.precio < 50)
  .map((p) => p.nombre);
// ['Teclado', 'Ratón', 'Casco']
```

Obtén, sin mutar productos:

1. Los nombres de los productos con stock.
2. El valor total del inventario (precio × stock).
3. Una copia ordenada por precio.
4. ¿Hay algún producto de más de 100 €?
5. Exporta cada solución como función de un módulo.